



Habib University
shaping futures

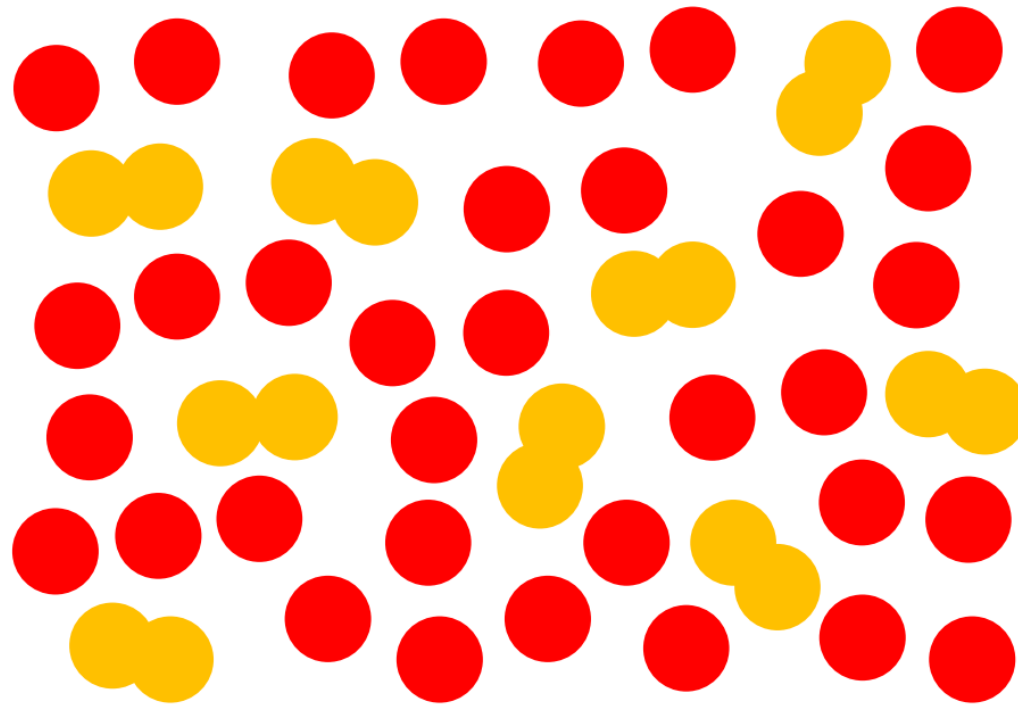
CS201 – Data Structures II

Lecture 06

Dr. Muhammad Mobeen Movania

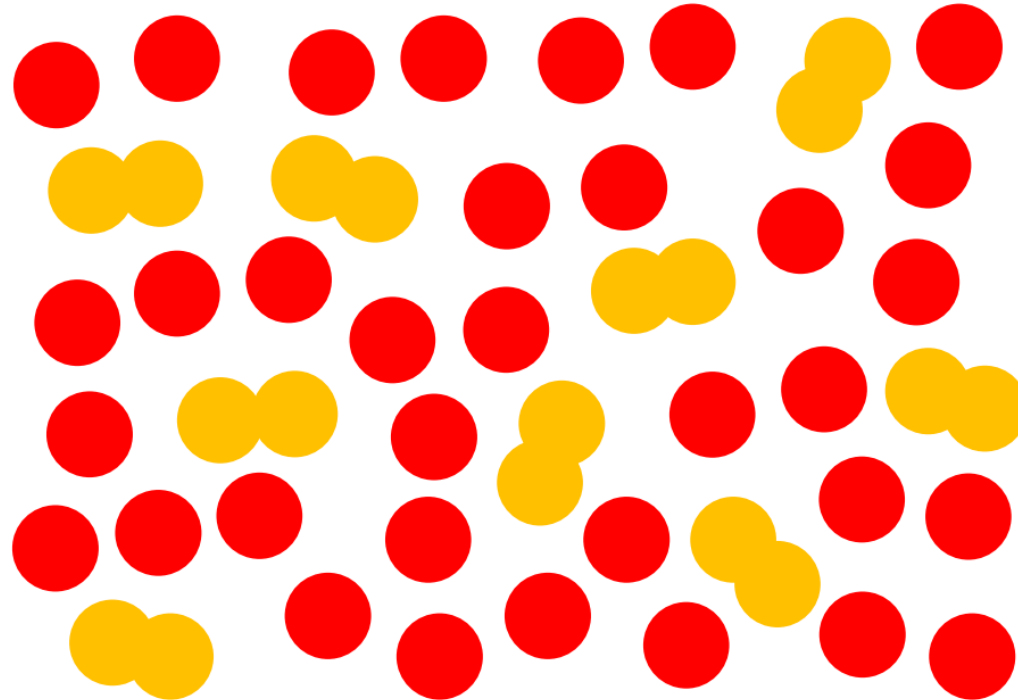
Spatial Hashing

- Problem:
 - Given n points, find all collisions of points with each other



Spatial Hashing

- Solution:
 - For all points $\mathbf{p}_i$: Find neighbors $\mathbf{p}_j$ such that $|\mathbf{p}_i - \mathbf{p}_j| \leq d$
 - Special case $d = 2r$: Find overlaps of particles with radius r



Spatial Hashing

- Brute force solution:

```
for all points  $\mathbf{p}_i$   
  for all points  $\mathbf{p}_j$   
    if  $|\mathbf{p}_j - \mathbf{p}_i| \leq d$   
      handleOverlap( $i, j$ )
```

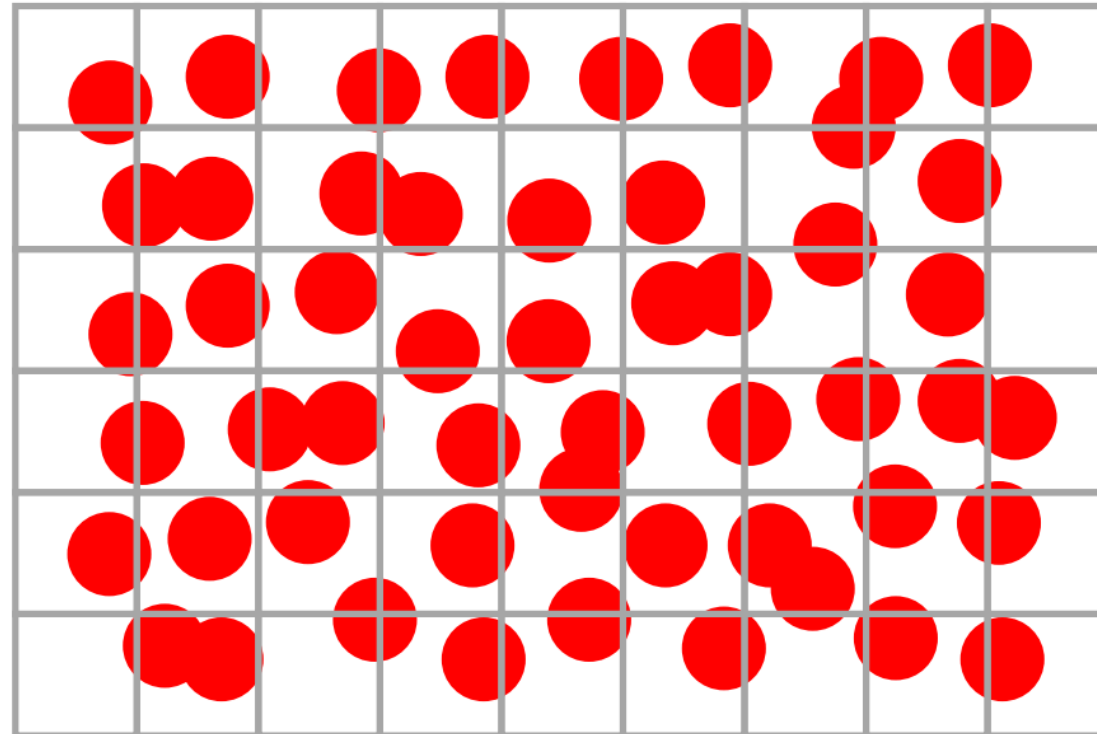
- Problems:
 - This is $O(n^2)$, for 100,000 points (common) we perform 10,000,000,000 test

Spatial Hashing

- In general for the complexity of a simulation algorithm:
 - $O(n)$ is good
 - $O(n \log n)$ is OK (i.e. sorting, $\log 100,000 \approx 17$)
 - $O(n^2)$ is not an option
- What should we do then?
 - Think of how we can use some sort of acceleration scheme

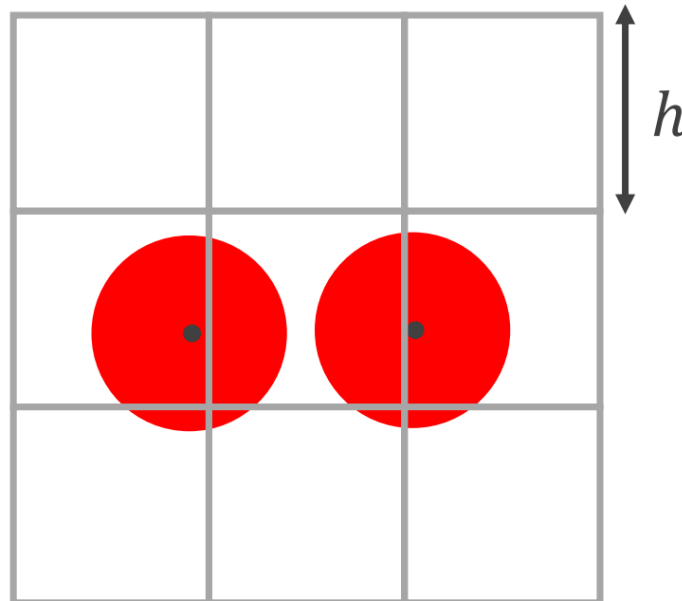
Spatial Hashing

- Key Idea:
 - Store particles in a regular grid
 - Instead of checking with all other particles, only check particles in the same and surrounding cells

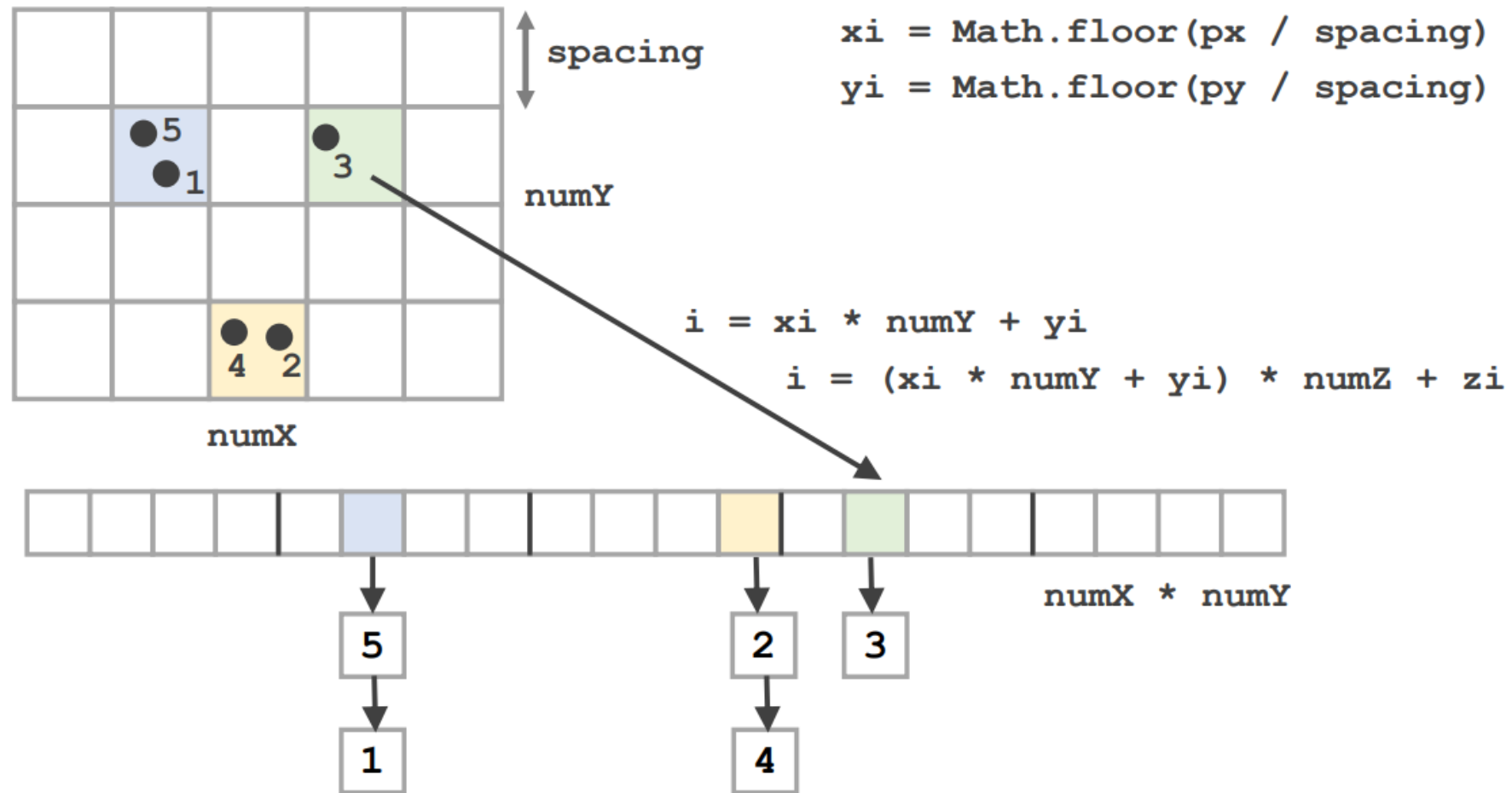


Spatial Hashing

- Grid layout
 - Store particles once where their center is located
 - If we choose $h = 2r$, we only need to check the containing and the surrounding cells
 - 9 in 2 dimensions, 27 in 3 dimensions



Storing the Grid



Hash function

- The hash function must return a value that distributes the cells evenly

```
hashCoords(xi, yi, zi) {  
    var h = (xi * 92837111) ^ (yi * 689287499) ^ (zi * 283923481);  
    return Math.abs(h) % this.numCells;  
}
```

- What should be the hash table size
 - Large hash table size: fewer collisions, fewer tests, faster
 - Choosing tableSize = numParticles